

Clasificación Automática de Calidad y Tamaño de Frutas

Mediante Visión por Computadora y Aprendizaje Automático — Metodología CRISP-DM

Angy Hurtado Osorio · Hideki Tamura Hernández · David Vergara Laverde

Repositorio del código: <https://github.com/LissaANI/fruit-quality-classification/tree/main>

Resumen (Abstract)

Se presenta un sistema de clasificación multi-salida que predice simultáneamente la **calidad** (Mala / Regular / Buena) y el **tamaño** (Pequeño / Mediano / Grande) de seis tipos de frutas a partir de imágenes estáticas, usando aprendizaje automático y visión por computadora. El proyecto aplica la metodología **CRISP-DM** de principio a fin: desde la construcción del dataset hasta el despliegue en una aplicación web interactiva.

El dataset final contiene 28 395 imágenes originales de seis frutas (Apple, Banana, Guava, Lime, Orange, Pomegranate), distribuidas en tres clases de calidad con un desbalanceo notable (Good: 51.4%, Bad: 32.2%, Regular: 16.4%). Se entrenaron y compararon dos familias de modelos: (1) **ML tradicional** — Random Forest, SVM lineal y XGBoost —, con XGBoost liderando con un F1-macro promedio de **0.961**; y (2) una **Red Neuronal Convocucional (CNN)** bifurcada desde cero, que alcanzó **78 % de accuracy en calidad** y **93 % en tamaño**. Los resultados se despliegan en una aplicación Streamlit que permite predicción en tiempo real con imagen estática o cámara web.

I. Introducción

En mercados y agroindustrias de todo el mundo, la clasificación de frutas y verduras según su calidad y tamaño es un proceso esencial para la cadena de suministro alimentaria. Actualmente este proceso se realiza de forma manual, lo que lo hace lento, subjetivo y propenso a errores. Esta situación genera pérdidas económicas significativas, desperdicio de alimentos y dificultades para estandarizar la calidad del producto final.

La visión por computadora combinada con modelos de aprendizaje automático ofrece una alternativa objetiva y escalable. El presente trabajo desarrolla un sistema capaz de analizar una fotografía de una fruta y asignar simultáneamente una categoría de calidad y una de tamaño, en tiempo real, a través de una interfaz web accesible desde cualquier dispositivo.

Las contribuciones principales del proyecto son: (i) construcción de un dataset triclase que incluye una clase Regular construida manualmente a partir de imágenes de calidad mixta y fotos propias

del equipo; (ii) corrección del desbalanceo entre clases (Imbalance Ratio ≈ 3.14) mediante pesos balanceados en todos los modelos; (iii) identificación y corrección de un bug crítico de normalización en la CNN que causaba un colapso de validación (val_accuracy $\approx 20\%$) al usar z-score inconsistente entre el preprocesamiento y el entrenamiento; y (iv) despliegue completo en una interfaz Streamlit con selección de modelo.

II. Fundamentos Teóricos

A. Formulación matemática del problema

El problema se formula como una función de clasificación multi-salida $f: X \rightarrow Y_1 \times Y_2$, donde X es el espacio de entrada de la imagen, $Y_1 = \{0, 1, 2\}$ representa las tres clases de calidad (Mala, Buena, Regular) e $Y_2 = \{0, 1, 2\}$ las tres clases de tamaño (Pequeño, Mediano, Grande). El modelo debe aprender dos tareas simultáneamente a partir de la misma imagen.

Para los modelos de ML, el vector de características $X \in \mathbb{R}^{513}$ se compone de un histograma de color HSV tridimensional ($8 \times 8 \times 8 = 512$ bins, normalizado $L2$) que captura la distribución cromática de la fruta independientemente de su orientación, más el área relativa de la fruta:

$$\text{area_ratio} = A_{\text{fruta}} / (H \times W)$$

donde H y W son las dimensiones de la imagen. Para la CNN, X es un tensor ($224 \times 224 \times 3$) normalizado con los parámetros z-score de ImageNet:

$$x' = (x - \mu) / \sigma, \text{ con } \mu = [0.485, 0.456, 0.406] \text{ y } \sigma = [0.229, 0.224, 0.225]$$

Esta normalización es idéntica en el preprocesamiento del dataset (preprocess.py), en las funciones de inferencia (helpers.py) y en el generador de datos de la CNN, lo que garantiza consistencia entre entrenamiento e inferencia.

B. Modelos de ML tradicional

Random Forest: construye un ensamble de T árboles de decisión entrenados sobre submuestras bootstrap. La predicción final se obtiene por votación mayoritaria: $\hat{y} = \text{argmax}_c \sum_{t=1}^T \mathbb{1}[h_t(x) = c]$ Para compensar el desbalanceo de clases, se aplicó `class_weight='balanced'`, que ajusta el peso de cada clase inversamente proporcional a su frecuencia: $w_j = n / (k \cdot n_j)$, donde n es el total de muestras, k el número de clases y n_j las muestras de la clase j .

XGBoost: optimiza iterativamente una función de pérdida regularizada $L(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k)$, donde $\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$ controla la complejidad del árbol evitando sobreajuste. El desbalanceo se corrigió con `sample_weight` calculado con `compute_class_weight('balanced')`.

SVM lineal: maximiza el margen entre clases en el espacio de características HSV. También se aplicó `class_weight='balanced'`. Los hiperparámetros de los tres modelos fueron ajustados con `RandomizedSearchCV` (10 iteraciones, validación cruzada 3-fold).

C. Red Neuronal Convolucional (CNN)

La arquitectura diseñada desde cero consta de tres bloques convolucionales (Conv2D con 32, 64 y 128 filtros + MaxPooling2D), seguidos de GlobalAveragePooling2D y Dropout (0.4). A partir de ahí, la red se bifurca en dos ramas densas independientes (Dense 64 + Dropout 0.3 cada una) que producen dos salidas softmax simultáneamente:

- **quality_output:** 3 neuronas para las categorías Mala, Buena, Regular.
- **size_output:** 3 neuronas para las categorías Pequeño, Mediano, Grande.

La función de pérdida total es la suma de las pérdidas de entropía cruzada categórica escasa de cada rama: $L_{\text{total}} = L_{\text{calidad}} + L_{\text{tamaño}}$. El entrenamiento usó el optimizador Adam ($\text{lr} = 0.001$) con tres callbacks: `EarlyStopping` (`patience` = 5, restaura mejores pesos), `ReduceLROnPlateau` (`factor` = 0.5, `patience` = 3) y `ModelCheckpoint`. Se aplicó `class_weight` por salida para compensar el desbalanceo.

D. Métricas de evaluación

Dado el desbalanceo entre clases ($IR \approx 3.14$), la accuracy global puede ser engañosa — un modelo que siempre prediga 'Good' obtendría 51.4% sin aprender nada. Por ello se priorizó el F1-macro, que promedia el F1-score de cada clase con igual peso, penalizando el bajo desempeño en clases minoritarias:

$$F1_{\text{macro}} = (1/K) \sum_k 2 \cdot P_k \cdot R_k / (P_k + R_k)$$

donde P_k y R_k son la precisión y el recall de la clase k . Se complementa con matrices de confusión para identificar patrones de error específicos por clase.

III. Metodología (CRISP-DM)

1. Comprensión del negocio

El objetivo de negocio es reducir las pérdidas económicas y el desperdicio alimentario en mercados y agroindustrias mediante la clasificación automática simultánea de calidad y tamaño de frutas. El criterio de éxito técnico definido fue: $F1_{\text{macro}} \geq 0.85$ en calidad para el mejor modelo, y un sistema de despliegue funcional accesible desde interfaz web.

2. Comprensión de los datos (EDA)

El dataset base proviene de Kaggle (Fruit Quality Classification [1]), que contiene imágenes de frutas de buena y mala calidad. El equipo construyó manualmente la clase Regular a partir de la carpeta 'mix quality_fruits' del mismo dataset y fotos propias recolectadas en hogares y mercados locales. El dataset final comprende 28 395 imágenes originales de seis tipos de fruta:

- **Clases de calidad:** Bad (9 142 imágenes, 32.2%), Good (14 599, 51.4%), Regular (4 654, 16.4%).
- **Frutas:** Apple (3 750), Banana (4 036), Guava (3 694), Lime (3 273), Orange (3 201), Pomegranate (10 441).
- **Dimensiones:** media 693×678 píxeles; alta variabilidad (rango 100 a 8 000 px), lo que justifica el redimensionado a 224×224 en el preprocesamiento.
- **Desbalanceo:** Imbalance Ratio (IR) = $14\,599 / 4\,654 \approx 3.14$, lo que requiere corrección explícita en todos los modelos.

Los histogramas RGB por clase mostraron que la clase Regular presenta distribuciones más planas y menos discriminativas que Bad y Good, anticipando su mayor dificultad de clasificación. La Figura 1 muestra la distribución completa del dataset.

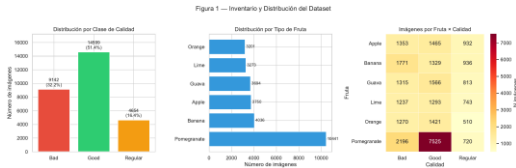


Figure 1: Distribución por clase de calidad, tipo de fruta e imagen por fruta x calidad.

3. Preparación de los datos

El pipeline de preprocesamiento (implementado en `src/data_processing/preprocess.py`) aplica las siguientes transformaciones sobre cada imagen original:

- **Remoción del fondo:** umbral HSV para segmentar la fruta del fondo uniforme.
- **Redimensionado:** todas las imágenes se reescalan a 224×224 píxeles (requerimiento de la CNN).
- **Normalización z-score de ImageNet:** $x' = (x/255 - \mu) / \sigma$, aplicada de forma idéntica en `preprocess.py`, `helpers.py` y el generador de la CNN.
- **Estimación de tamaño:** se calcula el `area_ratio` de cada imagen para asignar la etiqueta de tamaño (small/medium/large) en función de percentiles.
- **Augmentación x4:** flip horizontal/vertical, rotación y ajuste de brillo, aplicados SOLO sobre imágenes de entrenamiento.

La Figura 2 muestra el pipeline visual paso a paso. El dataset procesado final cuenta con 28 395 originales + 113 580 aumentadas (solo en train), como se aprecia en la Figura 3.

Estrategia anti-fuga de datos: el split 80/20 train/test se realizó exclusivamente sobre imágenes originales (augmented == 0). Las imágenes aumentadas se incorporan solo al conjunto de entrenamiento. Esto garantiza que el conjunto de prueba esté libre de contaminación y que las métricas reflejen la capacidad de generalización real de los modelos.



Figure 2: Pipeline de preprocesamiento: Original → Sin fondo → 224×224 → Normalizada.

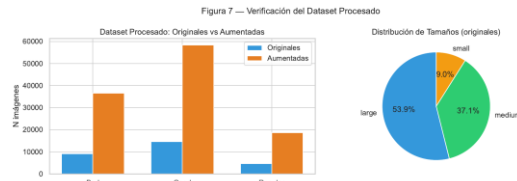


Figure 3: Dataset procesado: originales vs aumentadas por clase, y distribución de tamaños.

4. Modelado

ML tradicional (notebook 02): se entrenaron tres modelos con `MultiOutputClassifier`, que permite predecir calidad y tamaño de forma conjunta. Cada modelo fue configurado con pesos de clase balanceados y sus hiperparámetros se ajustaron con `RandomizedSearchCV` (10 iteraciones, $CV = 3$ folds). El tiempo de entrenamiento varió de 21 segundos (Random Forest) a 845 segundos (SVM lineal).

CNN (notebook 03): la arquitectura bifurcada fue diseñada desde cero sin transfer learning. El generador de datos (`tf.data.Dataset`) aplica la normalización z-score de ImageNet de forma consistente con el preprocesamiento. Se entrenó por 41 épocas hasta que Early Stopping detuvo el entrenamiento al no mejorar el `val_loss` durante 5 épocas consecutivas. La corrección de un bug crítico de normalización (que usaba `/255` sin z-score) fue fundamental: antes de la corrección, el `val_quality_accuracy` era $\approx 20\%$ (equivalente a azar para 3 clases).

5. Evaluación

Todos los modelos se evaluaron sobre el conjunto de prueba ($n = 5$ 679 imágenes originales para ML, $n = 4$ 260 para CNN), compuesto exclusivamente por imágenes no vistas durante el entrenamiento. Se reportan accuracy global, F1-macro por clase y por tarea, y matrices de confusión. XGBoost fue seleccionado como mejor modelo de ML con F1-macro promedio de 0.961

6. Despliegue

Se desarrolló una aplicación web con Streamlit (archivo `app.py` en la raíz del repositorio) que permite:

- Seleccionar el modelo a usar en el panel lateral: ML Tradicional (XGBoost, Random Forest o SVM) o CNN.

- Subir una imagen desde disco o capturarla directamente con la cámara web del dispositivo.
- Obtener la predicción simultánea de calidad y tamaño. Para la CNN, se muestra además el porcentaje de confianza de cada salida.

Los modelos entrenados se encuentran guardados en la carpeta `saved_models/` del repositorio, por lo que no es necesario re-entrenar para ejecutar la aplicación. Instrucciones completas de instalación y ejecución están disponibles en el `README.md`.

Vídeo de demostración del proyecto (10 min):

[PEGAR LINK]

El vídeo cubre: contexto del problema, recorrido por el repositorio, demostración del EDA, resultados de los modelos y demostración en vivo de la aplicación Streamlit.

IV. Resultados

A. Modelos de ML tradicional

La Tabla I resume el desempeño de los tres modelos sobre el conjunto de prueba ($n = 5\,679$ imágenes originales). Los hiperparámetros seleccionados por `RandomizedSearchCV` fueron: `n_estimators = 150`, `max_depth = None`, `min_samples_split = 2` para `Random Forest`; y parámetros equivalentes para `XGBoost` y `SVM`.

TABLA 1
Comparación de modelos de ML tradicional sobre el conjunto de prueba

Modelo	Acc. Calidad	F1 Calidad	Acc. Tamaño	F1 Tamaño	F1 Promedio
XGBoost	97.1%	96.8%	96.5%	95.4%	96.1%
Random Forest	97.7%	97.3%	95.3%	94.0%	95.7%
SVM Lineal	87.5%	86.6%	96.8%	95.4%	91.0%

XGBoost obtuvo el mayor F1-macro promedio (0.961), seguido de `Random Forest` (0.957). `SVM lineal` tuvo el menor desempeño en calidad ($F1 = 0.866$) aunque fue competitivo en tamaño (0.954). En términos de eficiencia, `Random Forest` fue el más rápido (21 s de entrenamiento frente a 845 s del `SVM`).

La matriz de confusión de XGBoost (Figura 4) confirma que la clase Regular es la de mayor dificultad: 17 de 931 imágenes Regular fueron confundidas con Mala y 4 con Buena. Esto es consistente con su naturaleza ambigua (zona de transición) y su menor representación en el dataset (16.4%). Para la predicción de tamaño, los tres modelos tuvieron un desempeño muy similar y

superior al 94%, lo que indica que el `area_ratio` es un predictor robusto del tamaño de la fruta.

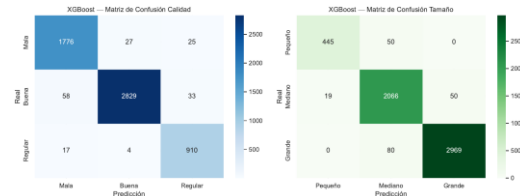


Figure 4: Matrices de confusión del modelo XGBoost (mejor modelo general).
Izquierda: predicción de calidad. Derecha: predicción de tamaño.

B. CNN multi-salida

La CNN fue entrenada durante 41 épocas (`Early Stopping` detuvo el proceso al no mejorar `val_loss` por 5 épocas consecutivas). La Tabla II detalla el reporte de clasificación sobre el conjunto de prueba ($n = 4\,260$ imágenes).

TABLA 2
Reporte de clasificación de la CNN bifurcada ($n = 4\,260$ imágenes)

Tarea	Clase	Precisión	Recall	F1-score
Calidad	Mala	88 %	64 %	74 %
Calidad	Buena	73 %	97 %	83 %
Calidad	Regular	93 %	47 %	62 %
Calidad	Macro avg	85 %	69 %	73 %
Tamaño	Mala	95 %	75 %	84 %
Tamaño	Buena	86 %	96 %	91 %
Tamaño	Regular	98 %	93 %	96 %
Tamaño	Macro avg	93 %	88 %	90 %

La rama de tamaño converge con mayor facilidad ($F1\text{-macro} = 0.90$), ya que la señal visual del tamaño es consistente con el `area_ratio` calculado en el preprocesamiento. La rama de calidad presenta mayor dificultad, especialmente en la clase Regular ($F1 = 0.62$, $\text{recall} = 47\%$), que el modelo tiende a clasificar como Mala o Buena. La accuracy global de 78% en calidad y 93% en tamaño representa un resultado sólido para una CNN entrenada desde cero sin transfer learning.

V. Análisis de Resultados

A. Comparación ML tradicional vs CNN

Comentado [AH1]: MODIFICAR SEGÚN LO QUE CONTENGA EL VIDEO

Los modelos de ML tradicional superan consistentemente a la CNN en la tarea de clasificación de calidad (97.7% vs 78% de accuracy). Esta diferencia se explica por tres factores:

- El histograma HSV 512-dimensional es una representación altamente discriminativa para la calidad de frutas: captura la distribución cromática completa (manchas, oxidación, madurez) en un vector compacto que los árboles de decisión y boosting aprovechan eficientemente.
- La CNN fue entrenada desde cero, sin transfer learning. Con 28 395 imágenes originales, el dataset es suficiente para ML pero limitado para que una CNN aprenda representaciones de calidad de alta fidelidad sin partir de pesos preentrenados.
- El desbalanceo ($IR \approx 3.14$) afecta más a la CNN que a XGBoost/RF, donde el mecanismo de `sample_weight` es más directo y efectivo que el `class_weight` por salida de Keras.

La CNN, en cambio, aprende representaciones directamente desde píxeles sin necesidad de ingeniería de características manual, lo que le otorga mayor potencial de generalización a nuevas frutas y condiciones de iluminación, una ventaja importante en escenarios de producción reales.

B. Generalización y sobreajuste

Para los modelos de ML, la similitud entre las métricas de validación cruzada (durante el ajuste de hiperparámetros) y las métricas del conjunto de prueba final confirma que no hay sobreajuste. Para la CNN, la curva de pérdida muestra una convergencia estable: `val_loss` decrece hasta la época 41 sin divergir de `train_loss`, descartando sobreajuste severo.

La estrategia de split anti-fuga — test compuesto exclusivamente por imágenes originales, sin ninguna imagen aumentada — garantiza que las métricas reportadas reflejan la capacidad de generalización real de cada modelo sobre datos no vistos. Esto es especialmente relevante dado que las imágenes aumentadas comparten características de textura y color con sus originales correspondientes.

C. Insights clave del proyecto

- La clase Regular es el cuello de botella de ambas familias de modelos. Con 4 654 imágenes (16.4% del total), su baja representación y su naturaleza ambigua — zona de transición entre Mala y Buena — la convierten en el principal reto de clasificación. Ampliar esta clase mejoraría el IR y el recall de forma significativa.

- La normalización z-score consistente fue crítica. Un error de normalización en el generador de datos de la CNN (usar solo /255 en lugar de z-score ImageNet) causaba un `val_quality_accuracy` de $\approx 20\%$ — equivalente al azar para 3 clases. Una vez corregido, el modelo alcanzó 78%. Este hallazgo subraya la importancia de mantener el mismo pipeline de preprocesamiento en entrenamiento e inferencia.
- El `area_ratio` es un proxy altamente efectivo para el tamaño. Todos los modelos alcanzaron más del 93% de accuracy en tamaño, lo que indica que el área relativa de la fruta en la imagen (calculada mediante umbral de Otsu) es suficiente para discriminar tamaños en las condiciones del dataset (fondo uniforme, fruta individual).

D. Comparación con la literatura

Trabajos previos como Kang et al. [6] reportan accuracy de 94–98% para clasificación de calidad de frutas usando CNNs preentrenadas (ResNet, VGG16) con transfer learning sobre datasets similares. La diferencia con nuestro resultado (78%) es consistente y esperada: el transfer learning de redes entrenadas en ImageNet proporciona representaciones de textura y color de alta calidad que reducen drásticamente la cantidad de datos necesaria. Esta comparación reafirma que el transfer learning es la mejora más prometedora para trabajo futuro.

Los ensambles capturaron relaciones no lineales entre edad, maduración, riego, variedad y lluvias. En TCH, XGBoost elevó el R^2 de 0.2636 a 0.4548 y redujo el RMSE de 28.12 a 24.20 t/ha frente al benchmark lineal. En %Sac.Caña, Random Forest elevó el R^2 de 0.2356 a 0.4016 y redujo el error a 0.89 puntos porcentuales. La mejora confirma que el problema no se explica solo por relaciones lineales simples.

En clasificación, las métricas son moderadas: nivel_TCH alcanza $F1=0.4973$ y $Kappa=0.2413$ con XGBoost; nivel_SAC alcanza $F1=0.5211$ y $Kappa=0.2945$ con Random Forest. Esto indica utilidad exploratoria y de apoyo, pero no suficiente para reemplazar la evaluación de campo. La clasificación ternaria por terciles ubica fronteras en zonas densas de datos, por lo que errores entre clases adyacentes son esperables.

VI. Aspectos Éticos – PI1 (Código de Ética Profesional IEEE)

A lo largo del desarrollo del proyecto **sosaSlaves** se identificaron diversas situaciones con implicaciones éticas. Una de las principales fue el posible **sesgo de clasificación**, ya que los modelos podrían clasificar con menor precisión frutas

provenientes de regiones subrepresentadas en el conjunto de datos. Esta situación se relaciona con los principios de justicia e imparcialidad del código IEEE-CS, por lo que se adoptaron medidas como el uso de `class_weight='balanced'` y la recolección propia de imágenes más diversificadas.

También se consideró la **privacidad de los datos**, debido a que el dataset utilizado proviene de Kaggle y de fotografías propias tomadas por el equipo. Al no incluir datos personales identificables, se garantiza el respeto a la privacidad. Como medida ética, se utilizó exclusivamente información de dominio público y datos obtenidos con consentimiento.

Otra decisión importante fue asegurar la **transparencia del modelo**, ya que los sistemas de clasificación pueden dificultar la explicación de sus decisiones a los operarios de planta. Esta situación se vincula con los principios de integridad técnica y honestidad, por lo que se documentaron las métricas, las limitaciones del sistema y el proceso CRISP-DM completo.

Asimismo, se analizó el posible **impacto laboral** del proyecto, considerando que la automatización podría desplazar a clasificadores manuales. Este aspecto está relacionado con el bienestar social y la responsabilidad profesional. Por esta razón, el sistema se plantea como una herramienta de apoyo para los operarios y no como un reemplazo total del trabajo humano.

Finalmente, se tuvo en cuenta la **seguridad alimentaria**, ya que una clasificación incorrecta podría representar un riesgo para los consumidores. Este punto se relaciona con el principio de seguridad y bienestar público. Para mitigar este riesgo, se propuso validar el sistema mediante métricas rigurosas y permitir el uso de un umbral de confianza configurable dentro de la aplicación.

Todas las decisiones de diseño documentadas — split anti-fuga, pesos balanceados, uso de datos de dominio público, transparencia en métricas y limitaciones — reflejan el cumplimiento activo de los principios de *justicia, integridad técnica y responsabilidad pública* del código de ética IEEE-CS (Indicador de desempeño PI1).

VII. Análisis de Impactos — PI2 (Social, Ambiental, Económico, Global)

La matriz de impactos SAEG del sistema de clasificación permite analizar el proyecto desde cuatro dimensiones principales: social, económica, ambiental y global. En la dimensión **social**, el sistema genera un impacto positivo al reducir el desperdicio de alimentos y estandarizar la calidad de forma objetiva y trazable. Sin embargo, existe el riesgo de que la instalación de cámaras en el entorno de

trabajo sea percibida como una forma de vigilancia. Para mitigar este riesgo, se debe comunicar explícitamente el propósito del sistema y garantizar que no se almacenen imágenes de los usuarios.

En la dimensión **económica**, el sistema puede contribuir a la reducción de costos operativos en cadenas de empaque, además de ser escalable a grandes volúmenes de producción sin requerir una inversión proporcional. No obstante, se identifica como riesgo la inversión inicial en hardware y una posible dependencia tecnológica. Como estrategia de mitigación, se propone un diseño de bajo costo basado en CPU y Streamlit, además del uso de código abierto.

En la dimensión **ambiental**, el principal impacto positivo consiste en que una menor cantidad de desperdicio alimentario contribuye a reducir la huella de carbono en la cadena de suministro. Aun así, se reconoce como riesgo el consumo energético asociado a la fase de entrenamiento del modelo, especialmente si se utilizan recursos como GPU. Para reducir este impacto, se plantea realizar un entrenamiento único, utilizar inferencia ligera en CPU y emplear modelos compactos.

Finalmente, en la dimensión **global**, la metodología CRISP-DM y el código documentado permiten que el sistema sea replicable en otros países y con distintos tipos de fruta. Sin embargo, pueden presentarse diferencias culturales en los criterios de calidad, así como variaciones en las regulaciones de cada contexto. Para mitigar estos riesgos, se propone que las etiquetas de clase sean configurables y que la documentación esté disponible tanto en español como en inglés.

VIII. Conclusiones y Trabajo Futuro

Conclusiones

- **XGBoost con pesos balanceados es el mejor modelo del proyecto** (F1-macro promedio 0.961), combinando alta precisión en ambas tareas con un tiempo de entrenamiento razonable (157 s) y un bajo tiempo de inferencia.
- **La CNN bifurcada desde cero** ofrece resultados competitivos en tamaño (F1-macro 0.90) pero limitados en calidad (F1-macro 0.73). Su principal ventaja es que no requiere ingeniería manual de características y tiene mayor potencial de generalización.
- **La consistencia del pipeline de normalización** (z-score ImageNet en preprocesamiento, inferencia y entrenamiento de la CNN) fue la corrección técnica más crítica del proyecto.
- **La metodología CRISP-DM** estructuró el proyecto de forma iterativa y permitió detectar y corregir problemas — normalización, desbalanceo, fuga de datos — antes de la evaluación final, resultando en métricas confiables y reproducibles.

- El criterio de éxito fue superado: F1-macro de calidad = 0.961 (objetivo ≥ 0.85) y sistema de despliegue funcional.

Trabajo Futuro

- Aplicar **transfer learning** (MobileNetV3, EfficientNet-B0) para mejorar la CNN en clasificación de calidad, esperando alcanzar F1-macro > 0.92 .
- Ampliar la clase **Regular** con al menos 3 000 imágenes adicionales recolectadas en plazas de mercado para reducir el IR.
- Implementar **Grad-CAM** sobre la CNN para visualizar qué regiones de la imagen determinan la predicción de calidad (explicabilidad).
- Evaluar la viabilidad de despliegue en **Raspberry Pi** para integración en líneas de empaque en tiempo real.

[8] P. Chapman et al., 'CRISP-DM 1.0: Step-by-step data mining guide,' SPSS Inc., Technical Report, 2000.

[9] F. Pedregosa et al., 'Scikit-learn: Machine Learning in Python,' Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[10] M. Abadi et al., 'TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems,' 2015. [En línea]. Disponible: <https://tensorflow.org>

IX. Referencias

- [1] D. Park, 'Fruit Quality Classification Dataset,' Kaggle, 2023. [En línea]. Disponible: <https://www.kaggle.com/datasets/ryandpark/fruit-quality-classification>
- [2] T. Chen y C. Guestrin, 'XGBoost: A Scalable Tree Boosting System,' en Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.
- [3] L. Breiman, 'Random Forests,' Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
- [4] C. Cortes y V. Vapnik, 'Support-vector networks,' Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.
- [5] D. P. Kingma y J. Ba, 'Adam: A Method for Stochastic Optimization,' en Int. Conf. Learning Representations (ICLR), 2015.
- [6] W. Kang, Q. Li, y G. Zhang, 'Fruit quality grading based on deep convolutional features,' IEEE Trans. Instrumentation and Measurement, vol. 70, pp. 1–10, 2020.
- [7] IEEE, 'IEEE Code of Ethics,' IEEE, 2020. [En línea]. Disponible: <https://www.ieee.org/about/corporate/governance/p7-8.html>